```
File  Edit  View  Search  Terminal  Help
# csh bshell.sh
Enter a Number
read: Command not found.
num: Undefined variable.
rem=: Command not found.
rem: Undefined variable.
# tcsh bshell.sh
Enter a Number
read: Command not found.
num: Undefined variable.
rem=: Command not found.
rem: Undefined variable.
# fish bshell.sh
bshell.sh (line 4): Missing end to balance thi
if [ $rem -eq  0 ]
^
<W> fish: Error while reading file bshell.sh
```

Figure 8: Failed execution of the script *bshell.sh*

```
File  Edit  View  Search  Terminal  Help
# csh cshell.sh
Enter a Number
777
Odd Number
# tcsh cshell.sh
Enter a Number
888
Even Number
```

Figure 9: Successful execution of the script *cshell.sh*

Now let us consider the shell script shown below called *cshell.sh*. This script is written in C shell syntax and is equivalent to the script *bshell.sh*, which checks whether the number entered through the keyboard is odd or even.

```
echo "Enter a Number "
set num = $<
@ rem = $num % 2
if ($rem ==  0) then
     echo "Even Number"
else
     echo "Odd Number"
endif
```

Figure 9 shows the output of the shell script *cshell.sh* when executed by the following two shells — the C shell and tcsh. From Figure 9, it is clear that the two shells were able to execute the shell script correctly without any errors. So, we can conclude that the C shell and tcsh have similar syntax (an obvious conclusion since tcsh is an upgraded version of the C shell). Notice that the C shell syntax has some similarity to the syntax of the C programming language. But this is not surprising because syntactic similarity with C was one of the design goals of the C shell. Figure 10 shows the execution of the shell script *cshell.sh* by the Bourne shell, Bash, KornShell, mksh, Z shell, and *fish*. From Figure 8, it is clear that all six of them failed to execute the script *cshell.sh*. Notice that the syntax of the *fish* shell script is different from all the others — but to what extent this is so, I don't know. I believe exploring the features of this shell called *fish* is an article for another day. But I urge you to explore this shell further. Both the shell and its manual look very creative.

```
File  Edit  View  Search  Terminal  Help
# sh cshell.sh
Enter a Number
cshell.sh: 3: cshell.sh: Syntax error: newline unexpected
# bash cshell.sh
Enter a Number
cshell.sh: line 2: syntax error near unexpected token `newline'
cshell.sh: line 2: `set num = $<'
# ksh cshell.sh
Enter a Number
cshell.sh: syntax error at line 3: `newline' unexpected
# mksh cshell.sh
Enter a Number
cshell.sh[2]: syntax error: unexpected 'newline'
# zsh cshell.sh
Enter a Number
cshell.sh:3: parse error near `\n'
# fish cshell.sh
cshell.sh (line 2): Expected a string, but instead found end of the statement
set num = $<
          ^
<W> fish: Error while reading file cshell.sh
```

Figure 10: Failed execution of the script *cshell.sh*

So, after our brief comparison of these Linux shells, it feels that for an average user of Linux, the differences in the behaviour of the shells may not be that visible. There are two reasons for this conclusion. First, whichever may be the shell used, the basic Linux commands are processed in the same way. Second, even though there are differences in the syntax of the shell scripts being used, these are not significant. Of course, there are differences between shells as far as features like string processing, remembering the command history, automatic command completion, automatic suggestions, and many more are concerned.

I am sure ardent fans of a particular shell might have a lot of reasons for their choice, but to a typical Linux user who is comfortable with one particular shell, the idea of migrating to another may not be that advisable due to the not-so-flat learning curve and the unexpected minor differences that might lead to errors. So, I believe, the moral of the story is that shell scripting is an important skill but if you are already comfortable with a shell then you probably don't have any reason to migrate to another.

At the same time, if you are an absolute beginner to Linux, then it might be a good idea to give some serious thought to the options before you choose your Linux shell, especially if you plan to use Linux seriously and not just dabble in it. END

By: Deepu Benson

The author is a free software enthusiast whose area of interest is theoretical computer science. The open source tools of his choice include ns-2 and ns-3. He maintains a technical blog at *www.computingforbeginners.blogspot.in*.